An Approach to Composing Concerns for an Executable Unified Domain Model

Van-Vinh Le[1,2] and Duc-Hanh Dang[2*]
[1]Vinh University of Technology Education
[2]University of Engineering and Technology, Vietnam National University, Hanoi
Email: levanvinh@vuted.edu.vn, hanhdd@vnu.edu.vn

***Abstract*—In Domain-Driven Design (DDD), the domain model plays a pivotal role. A domain model must precisely reflect the rules, behaviors, and logic of the problem domain. Current DDD methods typically focus on either the structural aspects or the behavioral aspects. However, other domain concerns, gathered from various stakeholders, are often overlooked. They need to be systematically defined and integrated to create an executable unified domain model. This paper presents a novel approach to composing concerns within an executable unified domain model. In our method, each concern is represented as a model in its corresponding DSL. We introduce an annotation-based mechanism for composing these concern models. The concern DSL combines both external and internal DSLs, ensuring the executability of the unified domain model. We have developed a support tool based on the EMF/ATL/Acceleo modeling framework and the DDD framework JDA. To demonstrate and evaluate the effectiveness of our approach, we applied the proposed method to several typical concern domains using the corresponding DSLs. The results demonstrate enhanced expressiveness, modularity, and executability of the unified domain model in the context of DDD.**

***Index Terms*—Domain-Driven Design; UML/OCL Metamodeling; Domain-Specific Languages; Model Transformations; Attribute-Oriented Programming.**

I. Introduction

In Domain-Driven Design (DDD) [1], [2], the domain model plays a pivotal role. A domain model must precisely, completely, and consistently reflect the rules, behaviors, and logic of the problem domain. All stakeholders, including domain experts and developers, should use a common *ubiquitous language* to describe, understand, and interact with that domain. Current DDD methods typically focus on either structural aspects, such as domain concepts and their relationships, or behavioral aspects, such as domain states and activities. However, other domain concerns—such as security, logging, and performance—are often overlooked, despite being crucial from a developer's perspective. These concerns, gathered from various stakeholders, need to be systematically defined and integrated to create an executable unified domain model.

Domain models are often represented using UML/OCL class diagrams [3] to capture domain concepts and the associations between them. The concept of the *domain model* is extended and given further emphasis in DDD [1]. In DDD, the domain model is used to capture additional domain concerns to fully reflect the problem domain. Many works [4], [5] focus on defining external DSLs to precisely represent domain models. However, these models often need to be complemented with operational semantics to provide additional viewpoints for developers and users, as envisioned in DDD. Several other works consider internal DSLs [6], [7] an effective approach for representing executable domain models. Specifically, the works in [8]–[12] use annotation-based language extensions of object-oriented programming languages to express domain constraints. Recent other works [13] aim to represent domain models in the context of modern software architectures, such as service-oriented and microservices architectures. Many works [14], [15] focus on using DSLs to specify concerns and emphasize the challenge of combining them to build a complete software model. Other studies [16]–[18] introduce methods for composing DSLs or metamodels and mapping source code to formal languages. However, these works have yet to define a mechanism for composing concerns within an executable unified domain model.

This paper presents a novel approach to achieving this goal. First, each concern is represented as a model in its corresponding DSL. Next, we introduce an annotation-based mechanism for composing these models. The underlying DSL combines both external and internal DSLs, ensuring the executability of the unified domain model. We have developed a support tool based on the EMF/ATL/Acceleo modeling framework [5] and the DDD framework JDA [12]. To demonstrate and evaluate the effectiveness of our approach, we applied it to concerns corresponding to the DSLs DCSL [12], MCCL [19], and AGL [20]. The results demonstrate enhanced expressiveness, modularity, and executability of the unified domain model in the context of DDD.

To summarize, the main contributions of this paper are as follows:

- A method called UDMM for composing concerns within an executable unified domain model for DDD.
- An annotation-based mechanism for composing DSLs.
- A support tool based on EMF and JDA that realizes the proposed method.

The rest of this paper is organized as follows: Section II motivates our work with examples. Section III provides an overview of our approach. Section IV explains how to specify and compose concerns. Section V discusses the support tool and includes a discussion. Section VI reviews related work. The paper concludes with a summary and suggestions for future work.

II. Motivating Example

This section motivates our work with an example problem domain corresponding to the course management system, CourseMan, adapted from the work in [12].

*Corresponding author

A. Example Domain Model

Figure 1 shows the UML/OCL class diagram [3] for the CourseMan domain. The underlying domain concepts are represented by three classes and two association classes. The classes `Student`, `CourseModule`, and `SClass` represent the students of the institution, the course modules offered by the institution, and the student class types from which students can choose, respectively. The association class `SClassRegistration` captures the many-to-many association between `Student` and `SClass`. Finally, the association class `Enrolment` captures the many-to-many association between `Student` and `CourseModule`.

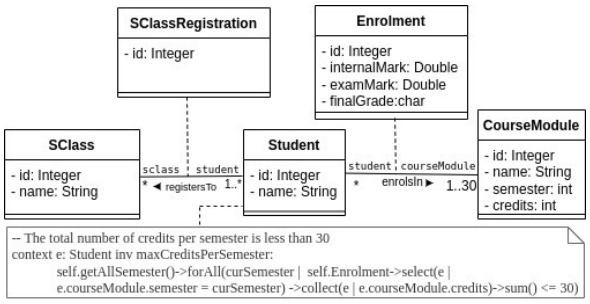

Fig. 1. UML/OCL class diagram for the CourseMan domain model.

To fully reflect such a problem domain, OCL constraints, as illustrated in Fig. 1, are often added to express additional restrictions on the domain model. These constraints primarily address concerns related to the structural aspects of the domain model. Other concerns, such as security, logging, and performance, should also be incorporated to create a complete domain model in DDD.

B. Composing Concerns in an Executable Domain Model

We might consider the CourseMan domain model, as illustrated in Fig. 1, as a unified domain model that combines the class diagram, as the core of the domain model, with the concern corresponding to the constraints expressed in the annotation boxes. These constraints are often expressed in either formal languages, such as OCL, or natural language.

As envisioned by DDD, such a unified domain model, in the form of a UML/OCL class diagram, should be executable, providing users or developers with suitable viewpoints extracted from the domain model. A possible solution would be to transform the domain model into an executable environment, such as Java. However, obtaining an executable viewpoint from the domain model remains a challenge, especially when the unified domain model incorporates additional concerns. This paper focuses on an approach to address this challenge.

III. Overview of Our Approach

This section explains our approach to composing concerns for an executable, unified domain model.

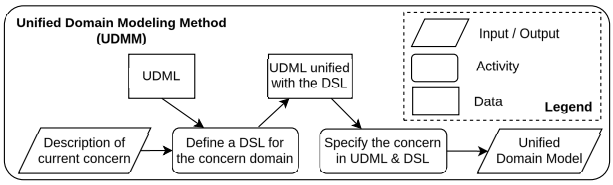

Fig. 2. Overview of the proposed approach.

As illustrated in Fig. 2, the proposed method, named UDMM (Unified Domain Modeling Method), consists of two main steps. First, to compose an underlying concern with the domain model in the so-called UDML, the designer must create a DSL, referred to as DSL_c, to represent the concern's domain. This involves defining meta-concepts within the domain based on its description, thereby creating a metamodel for the DSL_c. The DSL_c is then also unified within the UDML.

Second, the designer employs the UDML, unified with DSL_c, to represent the current concern, thereby creating a unified domain model. It is important to note that corresponding to DSL_c, the designer needs to define an annotation-based DSL (aDSL) [9], an internal DSL embedded in the host object-oriented programming language (OOPL). Each model in UDML, as an external DSL, corresponds to an equivalent model in the aDSL, meaning there is a one-to-one mapping between them. As explained in [18], the unified domain model, which can be viewed as an aDSL model, is executable.

We have defined the UDML metamodel, based on UML metaconcepts [3], as shown in Fig. 3. Four new metaconcepts—`DomainModel`, `Concern`, `Annotation`, and `Annotable`—have been added to facilitate the composition of concerns with the domain model and to unify DSL_c within UDML. Specifically, a `DomainModel` consists of `Annotable` elements and `Concerns`[1]. A `Concern` consists of `Annotations`. Each `Annotable` element is an instance of a metaclass from the UML metamodel for class diagrams, including `Package`, `Class`, `Property`, `Association`, and `Operation`.

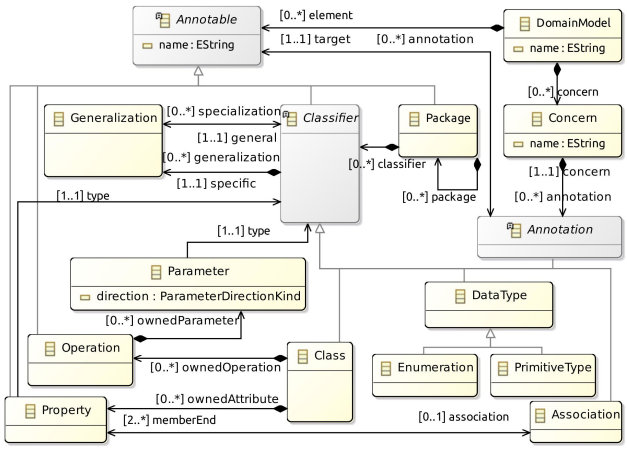

Fig. 3. The UDML metamodel for unified domain models.

IV. Specifying and Composing Concerns

This section aims to illustrate our approach to composing concerns, with a focus on the concern that can be expressed using the DCSL, as mentioned in [12].

A. Defining an External DSL for the Concern Domain

Capturing the concern domain. When building a domain model conforming to the metamodel UDML, i.e., in the form of a class diagram, we often need to define additional constraints to sharpen the domain. As shown in Table I, we focus on essential structural constraints for domain models, as explained in [12].

Defining the external DSL's abstract syntax. We aim to define a metamodel, as shown in Fig. 4, for the external DSL called DCSL to capture the underlying

[1] instances of the metaclass `Concern`

TABLE I
THE ESSENTIAL STRUCTURAL CONSTRAINTS (ADAPTED FROM [12])

Constraints	Type	Descriptions
Object mutability	Boolean	Whether or not the objects of a class are mutable [21]
Field mutability	Boolean	Whether or not a field is mutable (i.e. its value can be changed) [22]
Field optionality	Boolean	Whether or not a field is optional (i.e. its value needs not be initialised when an object is created) [22]
Field uniqueness	Boolean	Whether or not the values of a field are unique [22]
Id field	Boolean	Whether or not a field is an object id field [22]
Auto field	Boolean	Whether or not the values of a field are automatically generated (by the system) [22]
Field length	Non-Boolean	The maximum length (if applicable) of a field (i.e. the field's values must not exceed this length) [22]
Min value of a field	Non-Boolean	The min value (if applicable) of a field (i.e. the field's values must not be lower than it) [22]
Max value of a field	Non-Boolean	The maximum value (if applicable) of a field (i.e. the field's values must not be higher than this) [22]
Min number of linked objects	Non-Boolean	The minimum number of objects to which every object of a class can be linked [3]
Max number of linked objects	Non-Boolean	The maximum number of objects to which every object of a class can be linked [3]

concern domain. The metaconcepts `DomainModelDcsl`, `ConcernDcsl`, and `AnnotationDcsl` are defined to capture the DCSL concern domain and to integrate the concern into the UDML model. The metaconcepts

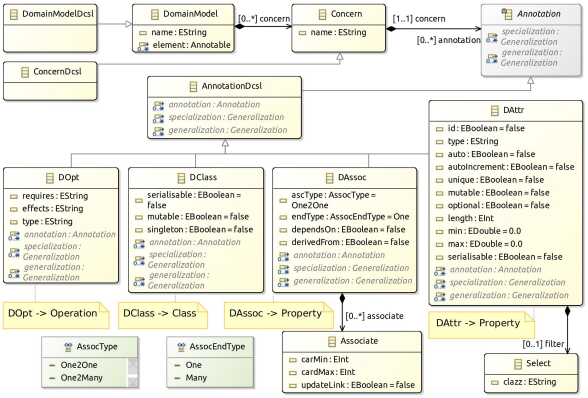

Fig. 4. The DCSL metamodel to capture the concern domain.

`DClass`, `DAttr`, `DAssoc`, and `DOpt` are defined as types of `AnnotationDcsl` and are mapped to `Class`, `Property`, `Property`, and `Operation`, respectively. We need to attach OCL constraints, such as the following, to the DCSL metamodel as well-formedness rules to maintain these mappings:

```
context DClass inv mapToCls:
    self.target.oclIsKindOf(Class)
```

The other concepts, along with the properties of all concepts, are defined to express the essential OCL constraints, which are summarized in Table I.

B. Mapping the External DSL to an Internal DSL

We aim to achieve the executability of unified domain models in UDML by mapping the external DSL for each concern domain to an aDSL [12], which is an internal DSL and a sublanguage of a host OOPL. To obtain such an aDSL, we first define a metamodel for its abstract syntax. We then establish a bidirectional transformation between the external DSL and the aDSL for a bijective mapping of their models. Finally, we define a transformation to generate code in the host OOPL from the concern model in aDSL. This transformation essentially involves two forms: the textual concrete syntax (reused from the host OOPL) and the abstract syntax of the aDSL.

Defining the aDSL's abstract syntax. Figure 5 shows the metamodel for the aDSL called DCSLcs, which corresponds to DCSL. The metaconcepts can be divided into two groups: The first group consists of the metaconcepts correspoding to the key concepts of the host OOPL, including `PackageCS`, `ClassCS`, `ClassifierCS`, `MethodCS`, and `FieldCS`. The second group consists of the metamodels corresponding to the DCSL metaconcepts.

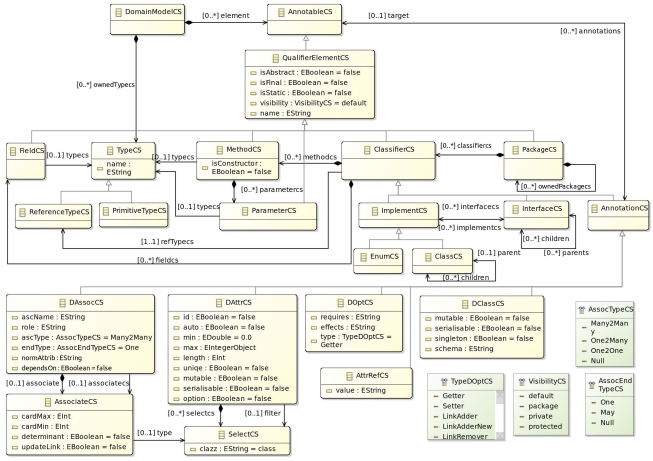

Fig. 5. The DCSLCS metamodel for the aDSL's abstract syntax.

Defining a bijective transformation between DCSL and DCSLcs models. We define a model-to-model transformation in a language like ATL [23] to maintain a bijective mapping between DCSL and DCSLcs models. The bijectiveness of this transformation is achieved through two straightforward one-to-one mappings. The first mapping connects the group of DCSLcs metaconcepts corresponding to the host OOPL with the corresponding UDML/DCSL metaconcepts, both of which capture the key metaconcepts for object-oriented models. The second mapping links the remaining DCSLcs concepts to their corresponding DCSL metaconcepts, representing the annotations for the underlying concern domain.

Generating the OOPL code from the aDSL model. We define a model-to-text transformation in a language like Acceleo [24] to generate code in the host OOPL (Java) from the aDSL model, which conforms to the abstract syntax of DCSLcs. In this way, we achieve the executability of the unified domain model within the host OOPL.

V. TOOL SUPPORT AND DISCUSSION

We have developed a support tool called UDMM, which consists of three main activities: (1) Defining and composing DSL_c, (2) Composing the concern into the UDML model, and (3) Generating software/prototype, as depicted in Fig.6. The first activity takes as input the concern domain specification (DSL_c) to define and compose DSL_c, resulting in the metamodel $eDSL_c$ (abstract syntax of the external DSL corresponding to the domain concern), metamodel $aDSL_c$ (metamodel and concrete syntax in textual form), ATL transformation $e2aDSL_c$, and Acceleo transformation $a2JavaDSL_c$. The

second activity uses the output of the first activity and the UDML model for the system, along with descriptions of the concern, to compose the concern into the domain model through two steps: Step 1 defines the DSL_c model ($eDSL_c$ and $aDSL_c$); Step 2 unifies the DSL_c model with the UDML model, resulting in a UDML model integrated with the concern. For the final activity, we rely on the

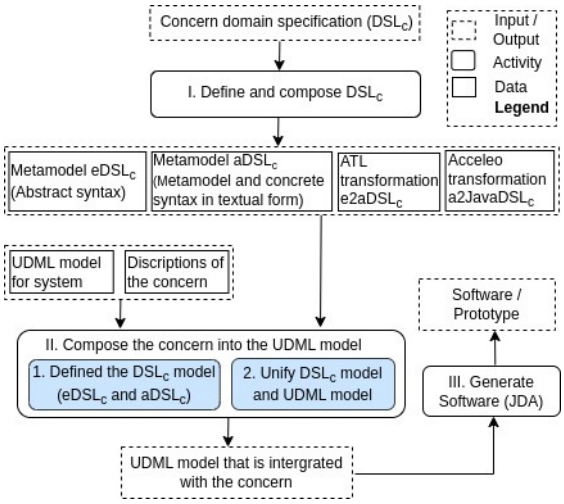

Fig. 6. The architecture of the support tool UDMM.

JDA tool [12] to generate software/prototype from the output of the second activity.

We have applied the proposed method to the case studies CourseMan and OrderMan. Figure 7 shows: (1) the UML class diagram with `DClass.mutable = true`, indicating the domain field name is mutable (`DAttr.auto = true`) and has a maximum length (`DAttr.length = 30`) in the CourseMan case study, which concerns domain specification; and (2) the resulting source code that addresses the concern's structural domain.

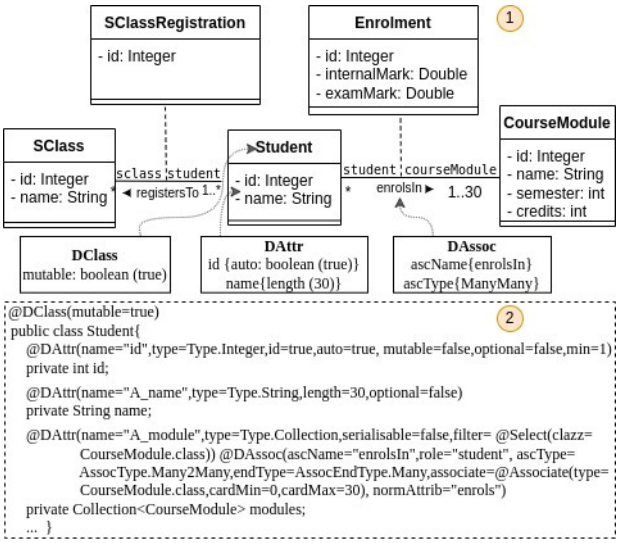

Fig. 7. An example of CourseMan software that addresses the concern's structural domain.

Discussion. The practical application of our method in developing software for real-world problem domains was successful. However, we discuss several threats to the validity of our method. First, the correctness of the method depends on the expressive capability of various concern domains, such as security, logging, and performance, as well as the application specification for integration into a coherent and executable UDM. The UDMM framework needs to be enhanced to capture different aspects of the concern domains. We believe this approach addresses multiple concern domains, such as security, logging, and performance, while directly supporting the perspectives of both developers and users, as envisioned in DDD. Second, to successfully achieve expressiveness, modularity, and executability of the UDM model, our method depends on the capability of the $e2aDSL_c$ and $a2JavaDSL_c$ transformations. Currently, correctness in this aspect is ensured by checking and reviewing the transformation rules, performing the converter tests, and conducting thorough tests for the transformation. We have carefully checked the rules in ATL and Acceleo for the transformation. Third, the software generated from the executable UDM in Java may not accurately reflect the input requirement specification if there are errors in the software/prototype generation using the JDA framework. We aim to improve and ensure accuracy by reviewing and checking the source code.

VI. Related Work

This section focuses on reviewing the following key research lines closely aligned with our contributions: (1) precise specification of domain models, (2) defining executable domain models, and (3) specifying and composing domain concerns. Building on these areas as the foundation of our research, we extend previous efforts in these directions.

Precise Domain Model Specification. The domain model is first employed within object-oriented analysis and design approaches to capture domain concepts from the problem domain and the associations between them, forming key abstractions that are mapped to entity classes in the object-oriented system. Within these approaches, domain models are often represented using UML/OCL class diagrams [3]. OCL, along with other formal languages, is used to precisely define constraints, including business rules, as restrictions on the domain model. The concept of the *domain model* is extended and given further emphasis within the DDD approach, introduced by Evans [1]. The domain model is used to capture not only the conceptual object model but also other concerns of the underlying domain, including business rules and domain knowledge, in order to fully reflect the problem domain. In DDD, the domain model becomes a unified representation from the perspectives of various stakeholders, including both domain experts and developers. Current works [4], [5], particularly within the context of Model-Driven Engineering (MDE), often focus on defining DSLs to precisely represent such domain models. The abstract syntax of a DSL is defined using a metamodel, while its concrete syntax can be defined in either graphical or textual form, as explained in [4], [5], [25]. In this research direction, to express the executability of the domain model, the corresponding DSL must have its operational semantics defined, as explained in [25]. The operational semantics can be defined using a transition system or through an equivalent transformation into an executable environment, such as a programming language. However, such a domain model inherently does not express executability and, therefore, does not directly support the perspectives of developers and users, as envisioned in DDD. Our work aims to define methods for specifying an executable unified domain model to address this limitation.

Defining Executable Domain Models. Current works consider internal DSLs [6], [7] an effective approach for representing executable domain models. Internal DSLs are embedded within an existing general-purpose programming language (GPL). Unlike external DSLs, which have their own syntax and operational semantics, internal DSLs leverage the syntax and semantics of the host language to provide domain-specific abstractions. This allows internal DSLs to serve as a bridge between technical implementation and business logic. Specifically, the works in [8]–[10] use annotation-based language extensions of object-oriented programming languages to express domain constraints. The work in [11] proposed an approach to bridge the gap between programming and modeling concerning annotation mechanisms, enabling the maintenance of the mapping between domain model representations at two levels: Java and UML profiles. Several other works [13] aim to represent domain models in the context of modern software architectures, such as service-oriented and microservices architectures. Our previous work [12] introduced an internal DSL along with a set of annotations to represent domain models. However, this research, along with the other works mentioned above, primarily focuses on the executability of domain models, while the challenge of specifying and composing concerns within the domain model is often overlooked.

Specifying and Composing Concerns. Separating and composing concerns are core principles in software development, aimed at managing complexity and increasing system flexibility. Many works [14], [15] focus on the approach of using DSLs to specify concerns and highlight the challenge of combining them to build a complete software model. The work in [16] identifies common mechanisms for the composition of DSLs, including referencing, extension, restriction, reuse, and embedding. Additionally, the work in [18] aims to establish the correspondence between source code annotations and formal languages, proposing an annotation-based approach for language composition. The work in [17] focuses on Cumbia, a metamodel-based platform for building applications that support multiple concern-specific languages. They propose a composition metamodel to characterize the composition and coordination of Cumbia models at the metamodel level. In contrast to these works, we aim to develop a mechanism for composing concerns in an executable unified domain model. This mechanism is based on the core idea of annotations and the one-to-one mapping between external DSLs and internal DSLs.

VII. Conclusion and Future Work

In this paper, we introduced a novel approach, UDMM, to address the challenge of composing concerns within an executable unified domain model in the context of DDD. By representing each concern as a model in its corresponding DSL and utilizing an annotation-based mechanism, our method ensures the seamless integration of both structural and behavioral aspects, alongside other domain concerns. The combination of external and internal DSLs further guarantees the unified model's executability. Through the development and application of a support tool based on the EMF/ATL/Acceleo framework and the JDA framework, we have demonstrated the enhanced expressiveness, modularity, and executability of the unified domain model.

Future work will focus on extending the capabilities of the UDMM tool and applying the approach to more complex domain models with additional typical concerns, further validating its practical utility.

References

[1] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley Professional, 2004.

[2] V. Vernon, Implementing domain-driven design, 1st Edition, Addison-Wesley Professional, 2013.

[3] OMG, Unified Modeling Language 2.5.1, Object Management Group, 2017.

[4] Andrzej Wąsowski, Thorsten Berger, Domain-Specific Languages Effective Modeling, Automation, and Reuse, Springer Cham, 2023.

[5] M. Brambilla, J. Cabot, M. Wimmer, Model-Driven Software Engineering in Practice, 2nd Edition, Morgan&Claypool, 2017.

[6] M. Fowler, Domain-Specific Languages, 1st Edition, Addison-Wesley Professional, 2010.

[7] A. Wąsowski, T. Berger, Internal Domain-Specific Languages, in: Domain-Specific Languages: Effective Modeling, Automation, and Reuse, Springer, Cham, 2023, pp. 357–394.

[8] Dan Haywood, Apache Isis - Developing Domain-Driven Java Apps, Methods & Tools: Practical knowledge source for software development professionals 21 (2) (2013) 40–59.

[9] J. Paniza, Learn OpenXava by example, 1st Edition, CreateSpace, 2011.

[10] I. Córdoba-Sánchez, J. De Lara, Ann: A domain-specific language for the effective design and validation of Java annotations, Computer Languages, Systems & Structures 45 (2016) 164–190.

[11] A. Bergmayr, M. Grossniklaus, M. Wimmer, G. Kappel, Leveraging annotation-based modeling with Jump, Software & Systems Modeling 17 (1) (2018) 65–89.

[12] D. M. Le, D.-H. Dang, V.-H. Nguyen, On domain driven design using annotation-based domain specific language, Computer Languages, Systems & Structures 54 (2018) 199–235.

[13] S. Kapferer, O. Zimmermann, Domain-specific Language and Tools for Strategic Domain-driven Design, Context Mapping and Bounded Context Modeling, in: Proc. 8th Int. Conf. Model-Driven Engineering and Software Development (MODELSWARD), Vol. 1, SciTePress, 2020, pp. 299–306.

[14] A. Van Deursen, E. Visser, J. Warmer, Model-driven software evolution: A research agenda, in: Proc. 1st int.l workshop on model-driven software evolution, MoDSE Nantes, France, 2007, pp. 41–49.

[15] J. B. Warmer, A. G. Kleppe, Building a Flexible Software Factory Using Partial Domain Specific Models, in: 6th OOPSLA Workshop on Domain-Specific Modeling, DSM, University of Jyvaskyla, 2006, pp. 15–22.

[16] M. Voelter, Language and IDE Modularization and Composition with MPS, Vol. 7680 of Lecture Notes in Computer Science (LNPSE), Springer, Berlin, Heidelberg, 2013, pp. 383–430.

[17] C. Rodríguez, M. Sánchez, J. Villalobos, Executable model composition: a multilevel approach, in: Proc. 2011 ACM Symposium on Applied Computing (SAC), 11, Association for Computing Machinery, 2011, pp. 877–884.

[18] M. Nosál', M. Sulír, J. Juhár, Language composition using source code annotations, Computer Science and Information Systems 13 (3) (2016) 707–729.

[19] D.M. Le, D.-H. Dang, V.-H. Nguyen, Generative software module development for domain-driven design with annotation-based domain specific language, Information and Software Technology 120 (2022) 106–239.

[20] D.-H. Dang, D. M. Le, V.-V. Le, AGL: Incorporating behavioral aspects into domain-driven design, Information and Software Technology 163 (2023) 107284.

[21] B. Liskov, J. Guttag, Program development in Java: abstraction, specification, and object-oriented design, Addison-Wesley Professional, 2000.

[22] J. A. Hoffer, J. George, J. A. Valacich, Modern Systems Analysis and Design, 7th Edition, Pearson, 2013.

[23] Frédéric Jouault a, Freddy Allilaire a, Jean Bézivin a, Ivan Kurtev, ATL: A model transformation tool, Science of computer programming 72 (1-2) (2008) 31–39.

[24] E. Syriani, L. Luhunu, H. Sahraoui, Systematic mapping study of template-based code generation, Computer Languages, Systems & Structures 52 (2018) 43–62.

[25] A. Kleppe, Software Language Engineering: Creating Domain-Specific Languages Using Metamodels, 1st Edition, Addison-Wesley Professional, 2008.